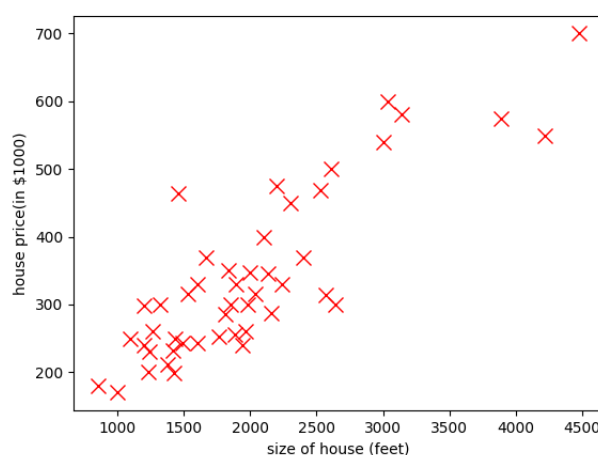


2 线性回归

2.1 线性回归模型

这里以房价预测为例，下图是对样本数据的可视化。



也可以使用表格的形式对数据进行可视化（左图），当数据输入特征更多时可以增加表格的列（右图）。

Living area (feet ²)	Price (1000\$s)	Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	400	2104	3	400
1600	330	1600	3	330
2400	369	2400	3	369
1416	232	1416	2	232
3000	540	3000	4	540
⋮	⋮	⋮	⋮	⋮

首先，介绍一些在机器学习中常用的术语和数学表示形式。

1. **数据集 (Dataset)**: 是指用于训练和评估机器学习模型的数据集合。数据集通常由多个样本组成，每个样本包含若干个属性或特征。在实验过程中，我们又会把数据集分成训练集 (Training set) 和测试集 (Testing set)，训练集用于训练模型，测试集用于评估模型。

2. **输入特征 x (input)**: 用于描述输入数据的变量或属性。

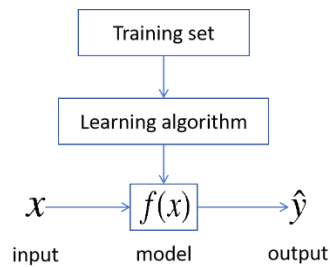
3. **输出目标 y (output)**: 希望模型预测的目标值 (标签)。

4. **m** : 数据集训练样本的总数。

5. **(x, y)** : 一个训练样本。

6. **$(x^{(i)}, y^{(i)})$** : 数据集中第 i 个样本。

机器学习的训练和工作的过程如下图所示，首先，将训练集给到训练算法中，经过训练得到一个公式 $f(x)$ ，这个公式就是所谓的模型 (model)，再将测试集的 x 输入到模型，模型会给出一个对目标 y 的估计或预测 $\hat{y}$ 。



那么，该如何表示公式 $f(x)$ ？我们需要对 $f(x)$ 进行一个假设（Hypothesis），假设 $f(x)$ 是一条直线：

$$f(x) = wx + b$$

学习算法通过不断学习来确定参数 w 和 b ，这就是机器学习的训练过程。我们接下来的课程会讲到的梯度下降法就是学习算法中最基础的一种。

本节的线性回归，这里的线性就是指公式是一条直线。

Part2-1 动手练

安装 matplotlib 库： `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple matplotlib`

导入库：

```
import matplotlib.pyplot as plt
import numpy as np
```

加载数据：

```
data = np.loadtxt('ex1data1.txt', delimiter=',', usecols=(0, 1))
X = data[:, 0]
y = data[:, 1] / 1000
m = y.size
print(m)
```

绘制散点图观察数据：

```
plt.plot(X, y, 'rx', ms=10)
plt.ylabel('house price(in $1000)')
plt.xlabel('size of house (feet)')
plt.show()
```

绘制直线 $f(x)$ 拟合数据：

```
w = 0.15
b = 20
x_fit = np.linspace(min(X), max(X))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')
plt.show()
```

2.2 代价函数 (Cost Function)

为了实现线性回归，第一个关键步骤是定义一个代价函数，代价函数能够告诉我们模型的运行情况，以便我们可以尝试让模型做的更好。前面介绍过用来拟合房价数据集的模型是线性函数 $f(x) = wx + b$ ， w 和 b 称为模型的参数 (parameter)，在机器学习中，模型的参数是可以在训练期间进行调整以改进模型的变量，参数 w 和 b 有时也被称为系数 (coefficient) 或权重 (weight)。

如何确定模型的参数就是我们想要的，这就需要定义一个代价函数对模型进行评估。代价函数由真实结果 y 与预测结果 $\hat{y}$ 进行比较得到。

首先，引入误差： $\hat{y} - y$

由于误差有正负之分，为了评价更公平，引入平方误差： $(\hat{y} - y)^2$

所有训练数据的平方误差为：
$$\sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

平方误差会随数据集增大而不断变大，为了构建一个不会随训练数据集大小变化而变化的代价函数，这里计算平均平方误差 (均方误差)：

$$\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

在实际使用中，代价函数还要再除以 2，额外除以 2 是为了让后面的计算更加简洁，

$$\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

通常代价函数用 $J(w, b)$ 表示，最终代价函数表示形式为：

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

还可以写成：

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m ((wx^{(i)} + b) - y^{(i)})^2$$

这个代价函数也称为均方误差代价函数。在机器学习中，我们要根据不同的应用使用不同的代价函数，均方误差代价函数目前是在线性回归中最常使用的。

为了实现线性回归，需要通过确定 w 和 b 的值来更好地拟合训练数据，而为了衡量 w 和 b 的选择与训练数据的拟合程度，用一个代价函数 $J(w, b)$ 衡量模型预测值与 y 的真实值之间的差异，线性回归会尝试找到使 $J(w, b)$ 尽可能小的 w 和 b 的值，因此目标函数可以写成：

$$\min_{w, b} J(w, b)$$

下面用一个简化版来举例说明如何找到参数的值，令 $b=0$ ，则线性函数表示为：

$$f_w(x) = wx$$

代价函数表示为：

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} - y^{(i)})^2$$

现在代价函数 $J(w)$ 只是 w 的函数，优化目标是找到使 $J(w)$ 最小化的 w 的值。

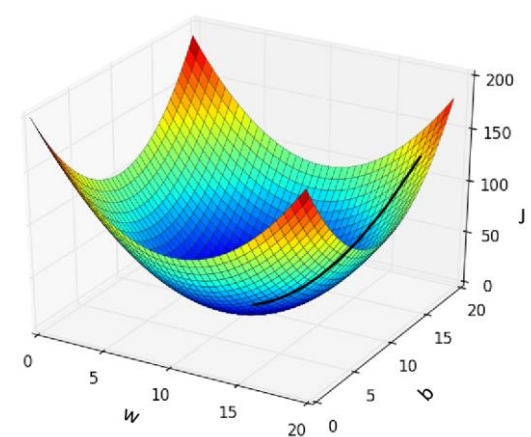
在这个简化的版本中，我们来观察当参数 w 选择不同的值时代价函数如何变化。假设我们只有 3 个训练数据 $(x=1, y=1)$, $(x=2, y=2)$, $(x=3, y=3)$ 。下面我们计算 w 取不同值时代价函数对应的值。

在更一般的情况下，参数包含 w 和 b ，线性回归的目标是要找到使代价函数最小化的 w 和 b 的值。

线性回归的一般形式可以写成：

模型：	$f_{w,b}(x) = wx + b$
参数：	w, b
代价函数：	$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$
优化目标：	$\min_{w, b} J(w, b)$

如下图所示，通常代价函数与参数 w 和 b 的关系呈现出一个“碗”的形式，碗的最底部对应的参数值使代价函数最小。



Part2-2 动手练

参数与代价函数可视化

```
import matplotlib.pyplot as plt

w = [0, 0.5, 1, 1.5, 2]
J_w = [2.5, 0.58, 0, 0.58, 2.5]

plt.plot(w, J_w)
plt.xlabel('w')
plt.ylabel('J')
plt.grid(True)
plt.show()
```

平滑曲线

安装 scipy 库: `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple scipy`

```
import matplotlib.pyplot as plt
from scipy.interpolate import make_interp_spline
import numpy as np

w = [0, 0.5, 1, 1.5, 2]
J_w = [2.5, 0.58, 0, 0.58, 2.5]

# 生成一个更细致的 x 轴 (用于插值)
x_new = np.linspace(0, 2, 100) # 100 个点从 0 到 2

# 使用样条插值
spl = make_interp_spline(w, J_w, k=3)
y_smooth = spl(x_new)

plt.plot(x_new, y_smooth)
plt.scatter(w, J_w, color='red') # 显示原始数据点
plt.xlabel('w')
```

```
plt.ylabel('J')
plt.grid(True)
plt.show()
```

2.3 梯度下降法（Gradient Descent, DG）

以上的例子只有一个或两个参数，在线性回归中，一旦参数增多，问题将变得更加复杂，我们无法通过人工的方式去确定参数的值，而是需要一种高效的算法，通过编程实现自动找到参数 w 和 b 的值。其中很重要的一个算法叫梯度下降法，这也是最基础的算法，后来很多优化算法都是梯度下降的变体。梯度下降法是一种用于尝试最小化任何函数的算法，不仅用于线性回归，也用于训练一些最先进的深度学习模型。

在使用线性回归时， w 和 b 的初始值是多少并不重要，所以一个常见的选择是将他们都设置为 0。在使用梯度下降法时，每次都稍微改变参数 w 和 b 的值，以尝试降低代价函数 $J(w, b)$ ，反复改变 w 和 b 的值，直到 $J(w, b)$ 稳定或接近最小值。

我们对梯度下降法做一个形象的比喻，假设我们站在一个山顶想快速下山，环顾四周发现最陡峭的方向是下山最快的，这在数学上称为最速下降方向，在该方向迈出的一步比在其他任何方向迈出一小步都快。迈出一小步后，我们再环顾四周，选择下山最快的方向再迈出一小步，这个过程就是梯度下降法的步骤。

接下来通过数学方法实现梯度下降，更新参数 w 的公式为：

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

这个表达式正在对参数 w 进行少量调整，其中 α 称为学习率， α 通常是 0~1 之间的一个小正数，可以是 0.01， α 的作用是控制“下坡”的幅度。当 α 很大时相当于走很大一步，当 α 很小时相当于走很小一步。 $\frac{\partial}{\partial w} J(w, b)$ 是梯度，即代价函数对 w 求偏导。

在我们的线性回归模型中不仅有参数 w 还有参数 b ，因此还要对 b 进行调整，与更新 w 的公式相似，更新参数 b 的公式为：

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

梯度下降法就是通过上面两个公式不断调整 w 和 b ，直到算法收敛

(convergence), 收敛意味着 $J(w, b)$ 取得最小值或接近最小值, 也就是说参数 w 和 b 不会再随着额外的步骤而发生太大的变化。

梯度下降法需要同时更新参数 w 和 b , 正确的步骤如下:

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$w = tmp_w$$

$$b = tmp_b$$

也就是说要先对两个参数进行计算然后再对参数进行更新。梯度下降法的过程就是重复执行参数的更新过程, 直到算法收敛。

梯度下降法的关键是学习率 α 的选择和梯度的计算。

学习率 α 的选择将对实现梯度下降的效率产生巨大的影响, 如果 α 选择不当, 梯度下降甚至可能根本不起作用。

梯度的计算结果为:

$$\frac{\partial}{\partial w} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\frac{\partial}{\partial b} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

事实证明，线性回归中使用均方误差作为代价函数不会存在多个局部最小值，参数 w 和 b 与 $J(w, b)$ 的函数图像呈现“碗”形，它具有单一的最小值，我们称这样的函数为凸函数。当在凸函数上使用梯度下降时，一个很好的特性是只要学习率 α 选择适当，总会收敛到全局最小值。

下面我们通过房价数据集观察梯度下降的过程。用下面这组数据模拟梯度下降法确定参数 w 和 b 值的大概过程。

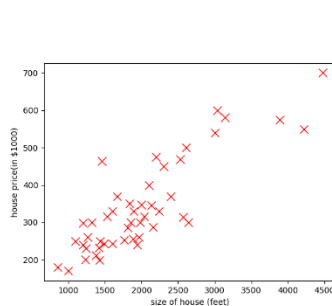
$w = -0.1, b = 500$

$w = -0.05, b = 300$

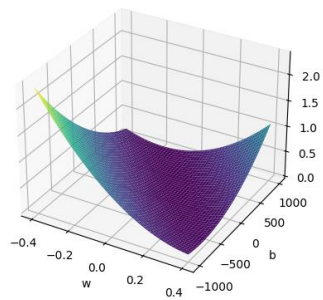
$w = 0.05, b = 100$

$w = 0.1, b = 50$

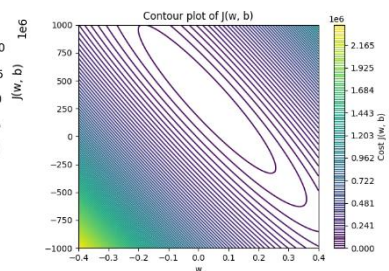
$w = 0.15, b = 20$



数据集



3D 视图



等高线图

Part2-3 动手练

定义代价函数

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 定义均方误差代价函数 J
def compute_cost(x, y, w, b):
    m = len(y) # 样本数量
    predictions = x * w + b
    cost = (1 / (2 * m)) * np.sum(np.square(predictions - y))
    return cost
```

加载数据

```
data = np.loadtxt('ex1data1.txt', delimiter=',')
x = data[:, 0]
y = data[:, 1] / 1000
```

绘制散点图


```
plt.figure(1)
plt.plot(x, y, 'rx', ms=10)
plt.ylabel('house price (in $1000)')
plt.xlabel('size of house (feet)')
plt.title('Scatter plot of house sizes vs prices')
```

绘制拟合直线

```
w = -0.1
b = 500
x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')

w = -0.05
b = 300
x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')

w = 0.05
b = 100
x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')

w = 0.1
b = 50
x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')

w = 0.15
b = 20
x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')

plt.show()
```

绘制 3D 视图

```
# 初始化参数 w 和 b 的范围
w_values = np.linspace(-0.4, 0.4, 100)
b_values = np.linspace(-1000, 1000, 100)

# 初始化一个空的代价函数值矩阵
J_values = np.zeros((len(w_values), len(b_values)))
```

```

# 计算所有 w 和 b 组合下的代价函数值
for i, w in enumerate(w_values):
    for j, b in enumerate(b_values):
        J_values[i, j] = compute_cost(x, y, w, b)

# 创建第二个图窗用于绘制 3D 表面图
fig = plt.figure(2)
ax = fig.add_subplot(111, projection='3d')
X, Y = np.meshgrid(w_values, b_values)
ax.plot_surface(X, Y, J_values, cmap='viridis', alpha=0.5)

# 设置标签
ax.set_xlabel('w')
ax.set_ylabel('b')
ax.set_zlabel('J(w, b)')
ax.set_title('Surface plot of J(w, b)')

# 假设您有几个特定的点想要在 3D 图上绘制
specific_points = [
    (-0.1, 500, compute_cost(x, y, -0.1, 500)),
    (-0.05, 300, compute_cost(x, y, -0.05, 300)),
    (0.05, 100, compute_cost(x, y, 0.05, 100)),
    (0.1, 50, compute_cost(x, y, 0.1, 50)),
    (0.15, 20, compute_cost(x, y, 0.15, 20))
]
specific_x, specific_y, specific_z = zip(*specific_points)
ax.scatter(specific_x, specific_y, specific_z, c='r',
marker='x', s=50)
plt.show()

```

绘制等高线图

```

# 创建第三个图窗用于绘制等高线图
fig, ax = plt.subplots(num=3)
X, Y = np.meshgrid(w_values, b_values)
CS = ax.contour(X, Y, J_values, levels=np.linspace(0,
np.max(J_values), 100), cmap='viridis')

# 添加颜色条
cbar = fig.colorbar(CS)
cbar.set_label('Cost J(w, b)')

# 设置标签和标题
ax.set_xlabel('w')
ax.set_ylabel('b')

```

```

ax.set_title('Contour plot of J(w, b)')

scatter_x = [-0.1, -0.05, 0.05, 0.1, 0.15]
scatter_y = [500, 300, 100, 50, 20]
ax.scatter(scatter_x, scatter_y, c='r', marker='x', s=50)

plt.show()

```

下面用另外一个数据集来实现梯度下降算法，假设现在你要在一个城市运营一辆快餐车，任务的目标是根据城市的人口数量预测利润，数据集的第一列表示人口数量，第二列表述利润。

```

# 定义梯度下降法
def gradientDescent(x, y, w, b, alpha, iters):
    cost = np.zeros(iters) # 初始化代价函数值为 0 数组，元素个数为迭代次数

    for i in range(iters):
        error = (x * w + b) - y
        tmp_w = w - (alpha / len(x)) * np.sum(error * x)
        tmp_b = b - (alpha / len(x)) * np.sum(error)
        w = tmp_w
        b = tmp_b
        cost[i] = compute_cost(x, y, w, b) # 计算每一次的代价函数

    return w, b, cost

# 加载数据
data = np.loadtxt('ex1data2.txt', delimiter=',')
x = data[:, 0]
y = data[:, 1]

# 参数初始化
alpha = 0.01
iters = 1500
w = 0
b = 0
# 执行梯度下降法
w, b, cost = gradientDescent(x, y, w, b, alpha, iters)

# 数据可视化
plt.plot(x, y, 'rx', ms=10)
plt.ylabel('profit')
plt.xlabel('population')

```

```

x_fit = np.linspace(min(x), max(x))
y_fit = w * x_fit + b
plt.plot(x_fit, y_fit, '-')
plt.show()

# 绘制代价函数随迭代次数的变化
plt.plot(range(iters), cost, '-')
plt.xlabel('iterations')
plt.ylabel('cost')
plt.show()

```

2.4 多元线性回归（Multiple Linear Regression）

通常的任务中不仅有一个特征，而是有多个特征，有多个特征的线性回归问题称为多元线性回归。

在前面介绍的线性回归问题中只有一个输入特征 x （房屋面积），任务的目标是根据 x 预测 y （房屋价格），因此线性回归模型是 $f_{w,b}(x) = wx + b$ 。现在我们不仅知道房屋面积，而且知道卧室数量、楼层、房龄，这给预测房价任务提供了更多信息，用 x_1, x_2, x_3, x_4 分别表示这四个特征。

此处，再引入几个新的数学表示形式：

1. x_j 表示第 j 个特征。
2. n 表示特征的数量。
3. $\vec{x}^{(i)}$ 第 i 个训练样本的特征向量。
4. $x_j^{(i)}$ 表示第 i 个训练样本的第 j 个特征。

当有四个特征时，模型定义为：

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

当有 n 个特征时，模型可以写成如下形式：

$$f_{\vec{w},b}(\vec{x}) = \sum_{j=1}^n w_jx_j + b$$

还可以简化写成：

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

这里 $\vec{w}, \vec{x}$ 都表示行向量，符号 $\bullet$ 表示两个向量对应元素相乘并将结果相加的运算，

$$\vec{w} = [w_1 \ w_2 \ \dots \ w_n]$$

$$\vec{x} = [x_1 \ x_2 \ \dots \ x_n]$$

2.5 向量化 (Vectorization)

向量化能使算法的代码更加简洁，并且能更好地利用硬件的并行能力，使代码执行速度更快。假设我们的多元线性回归问题中有 4 个特征，则

$$n=4$$

$$\vec{x} = [x_1 \ x_2 \ x_3 \ x_4]$$

$$\vec{w} = [w_1 \ w_2 \ w_3 \ w_4]$$

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

在线性代数中，向量的下标从 1 开始，而当我们编写代码时，通常用数组定义向量，数组的索引是从 0 开始的。如果不使用向量化计算 $f_{\vec{w},b}(\vec{x})$ ，代码如下所示：

```
w = [0.1, 0.2, 0.3, 0.4]
b = 10
x = [100, 200, 300, 400]

f = w[0]*x[0] + w[1]*x[1] + w[2]*x[2] + w[3]*x[3] + b
```

当 n 很大时，这样编写代码是低效的，可以使用循环的方式来编写代码。

```
w = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
b = 10
x = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]

f = 0
for j in range(len(x)):
    f += w[j] * x[j]
f = f + b
```

这样写代码虽然看起来简洁一些，但是执行效率依然很低，因为代码还是一步一步地计算乘法再计算加法， n 有多大代码就要循环多少次。下面我们来看量化的代码怎么实现。

向量化的数学表达形式为： $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

代码实现如下：

```
import numpy as np

w = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
b = 10
x = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]

f = np.dot(w, x) + b
```

这是 `numpy` 库实现向量化的代码，当 n 很大时，这种方式会比前两种写法运行效率快得多。因此，实现向量化有两个好处，一是使代码更简洁，二是代码的运行效率更快。向量化背后的本质是利用硬件的并行能力，让所有的乘法运算

并行执行，然后再将结果相加。

Part2-4 动手练

对比向量化和非向量化的运行时间

```
import numpy as np
import time

# 设置向量的大小
vector_size = 100000

# 生成两个随机一维向量
a = np.random.rand(vector_size)
b = np.random.rand(vector_size)

# 向量化
start_time_vectorized = time.time()
c_vectorized = np.dot(a, b)
end_time_vectorized = time.time()
vectorized_time = end_time_vectorized - start_time_vectorized

# 非向量化向量乘法（使用循环）
start_time_non_vectorized = time.time()
c_non_vectorized = 0.0
for i in range(vector_size):
    c_non_vectorized += a[i] * b[i]
end_time_non_vectorized = time.time()
non_vectorized_time = end_time_non_vectorized - start_time_non_vectorized

# 输出结果
print(f"向量化向量乘法时间: {vectorized_time:.6f} 秒")
print(f"非向量化向量乘法时间: {non_vectorized_time:.6f} 秒")
print(f"性能提升: {non_vectorized_time / vectorized_time:.2f} 倍")
```

2.6 多元线性回归中的梯度下降

参数: $\vec{w} = [w_1, w_2, \dots, w_n], b$

模型: $f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

代价函数:
$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$

梯度下降过程:

```
repeat {
     $w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$ 
     $b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$ 
}
```

2.7 特征缩放 (Feature Scaling)

在多元线性回归模型中，不同的特征通常拥有不同的测量单位和取值范围。特征值的尺度差异会对模型的性能产生不利影响，尤其在使用梯度下降法求解时，尺度的不一致可能导致梯度下降的过程变得异常缓慢，甚至陷入局部最优解。因此，特征缩放通过调整数据尺度，使得所有特征对模型训练的影响是平等的。

举一个具体例子来说明特征缩放的意义，这里我们使用房屋面积和卧室数量预测房价，房屋面积 x_1 的取值范围是 300~2000，卧室数量 x_2 的取值范围是 1~5，这里 x_1 很大而 x_2 很小，当特征取值范围很大时，模型可能选择的参数很小，反之当特征取值范围很小时，参数很大。那么代价函数的等高线视图如图(a)所示，横轴 w_1 的范围比较小，纵轴 w_2 的范围比较大。因为 w_1 的微小变化对预测价格会产生很大的影响，而 w_2 需要更大的变化才能对预测价格产生同样的影响，所以在使用梯度下降法时，寻找最小值的路径可能会来回波动，如图(b)所示。为了避免这种情况，使用特征缩放让特征的取值范围都在相同的区间，比如 0~1，这样得到的代价函数等高线视图如图(c)所示，使用梯度下降法寻找最小值的速度也比较快。

下面介绍两种实现特征缩放的方法。

最小-最大归一化 (Min-Max Normalization):

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

其中, x_{min} 是特征的最小值, x_{max} 是特征的最大值, x 是原始的特征值, x' 是归一化后的特征值, 这种方式所有特征分量都将缩放范围 0~1。

Z-Score 归一化 (Z-Score Normalization), 这种方法不保证归一化到[0, 1]范围内, 在平均数之上的数会得到一个正的标准值, 在平均数之下的数会得到一个负的标准值:

$$x' = \frac{x - \mu}{\sigma}$$

其中, μ 是特征 x 的均值, σ 是特征 x 的标准差。

Part2-5 动手练

安装 pandas 库: `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple pandas`

Pandas 库为我们提供了提供丰富的数据操作功能, 如数据清洗、缺失值处理、数据转换、数据筛选、分组、聚合等。

导入库

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

加载数据

```
data=
pd.read_csv('ex1data3.txt', names=['square', 'bedrooms', 'price'
'])
```

特征缩放

```
# Z-score 特征缩放
data = (data - data.mean()) / data.std()
```

数据预处理

为了实现统一的矩阵运算, 我们通常在数据的最前面加一列全为 1 的数据, 这样参数 b 就能与参数 w 合成一个向量, 用统一的方式进行计算。

```
data.insert(0, 'Ones', 1)      # 在第 0 列插入表头为“ONE”的列, 数值为 1
cols = data.shape[1]           # 获取表格 df 的列数
X = data.iloc[:, 0:cols-1]     # 取除最后一列外, 取其他列的所有行
y = data.iloc[:, cols-1:cols]  # 取最后一列的所有行, 即目标 y (利润)
X = X.values
y = y.values.flatten()
```

定义代价函数 (向量化)


```
def compute_cost (X,y,theta):
    inner=np.power(np.dot(X, theta)-y,2)
    # np.power(A,B)    ## 对 A 中的每个元素求 B 次方
    return np.sum(inner)/(2*len(X))
```

定义梯度下降函数

```
def gradientDescent(X, y, theta, alpha, iters): # alpha 是学
习率, iters 为迭代次数
    m, n = X.shape
    cost = np.zeros(iters) # 初始化代价函数值为 0 数组, 元素个数为迭
代次数

    for i in range(iters):
        y_pred = np.dot(X, theta)
        dw = (1 / m) * np.dot(X.T, (y_pred - y))
        theta -= alpha * dw

        cost[i] = compute_cost(X, y, theta) # 计算每一次的代价函
数

    return theta, cost
```

初始化参数

```
theta = np.zeros(3)    # [b, w1, w2]
alpha = 0.01
iters = 1500
```

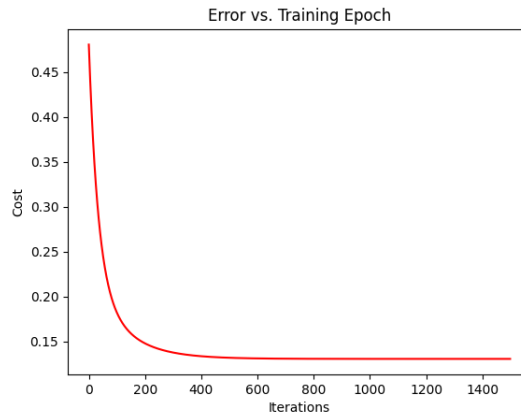
使用梯度下降法训练模型并将结果可视化

```
# 执行梯度下降函数
theta, cost = gradientDescent(X, y, theta, alpha, iters)

# 训练过程可视化
plt.plot(np.arange(iters), cost, 'r')
plt.xlabel('Iterations')
plt.ylabel('Cost')
plt.title('Error vs. Training Epoch')
plt.show()
```

2.8 学习曲线 (Learning Curve)

在使用梯度下降法时, 如何判断算法运行良好, 也就是判断算法是否正在帮助我们寻找使代价函数最小的参数。可以通过观察学习曲线来进行判断, 学习曲线即训练过程中代价值的变化曲线, 曲线的横轴表示算法迭代的次数, 纵轴表示代价值, 我们在前面的实验中已经画过学习曲线, 如下图所示。



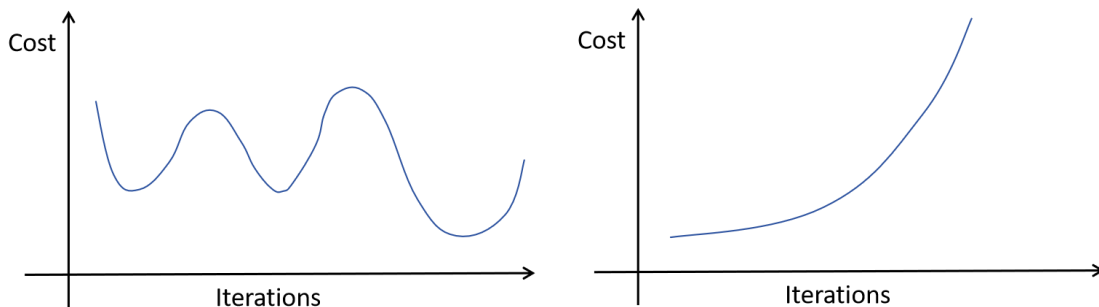
如果算法运行良好，则每次迭代代价函数的值都在减小。相反，如果某次迭代后代价函数的值增大了，这通常意味着学习率 α 选择不当或者代码存在问题。

通过观察学习曲线还能看出迭代多少次时曲线不再下降，此时算法基本就收敛了。实际上，我们很难提前知道梯度下降迭代多少次才能收敛，有的问题可能几十次就收敛了，而有的问题可能需要几千次，通过观察学习曲线可以判断什么时候停止训练。

选择学习率

梯度下降更新公式中很重要的一个参数是学习率 α ，选择合适的 α 对算法运行很重要，下面介绍如何选择 α 。

观察学习曲线，如下图所示，如果曲线上下波动或者呈上升趋势，意味着代码有误或者 α 过大，此时应该减小 α 的值。



通常在选择 α 时，一开始要选择一个非常非常小的值，如果在 α 非常小的情况下学习曲线仍然不是一条下降的曲线，那么可能代码存在问题，比如参数更新公式中的减号写成了加号。如果此时学习曲线是一条下降的曲线，那么我们就可以尝试增大 α 。通常选择 α 尝试以下几个值，然后找到最大的合理值。

0.001 0.01 0.1 1 2... ..

比如一开始 α 选择 0.001，观察学习曲线是下降的，再选择 0.01，如果学习曲线还是下降的，可以选择 0.1，如果此时学习曲线不再是下降的，说明 α 取 0.1 太大了，我们再尝试选择 0.01~0.1 之间的值来观察，直到找到 α 最大的合理值。

作业：基于 part2-5 的实验，选择不同的学习率观察学习曲线。

2.9 特征工程 (Feature Engineering)

特征工程是机器学习中一个很重要的手段。比如在房价预测案例中，数据提供给我们的两个特征是房屋的长和宽，通过这两个特征我们可以构建一个模型进行房价预测，

$$y = w_1x_1 + w_2x_2 + b$$

我们还可以利用现有的两个特征构建出一个新的特征房屋的面积 x_3 ，然后利用这 3 个特征构建一个模型进行房价预测，

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

这个模型也许能够更好地对房价进行预测。

特征工程是机器学习中的一个重要概念，它是从原始数据中提取和构建对模型预测任务更有用的特征的过程。常见的特征工程方法包括：特征选择和特征构造。

上面例子利用现有特征构建新的特征也就是特征构造，在实际问题中，不仅可以利用已知的数据，还可以利用特征工程定义新的特征来帮助我们获得更好的模型。

2.10 多项式回归 (Polynomial Regression)

将线性回归和特征工程的思想结合起来能得到多项式回归。多项式回归通过使用多项式函数来拟合数据，而不仅仅是线性函数。多项式回归通常用于处理数据与变量之间的非线性关系，这种关系不能用线性回归来准确建模。